

PEP 8 SUMMARY

- **PEP 8** is the official Python style guide.
- It tells us **how to write clean, readable Python code**.
- It does **NOT** change how Python works — it only improves readability and professionalism.

Why PEP 8 Matters

- Write readable and consistent code
- Follow industry standards
- Work in teams using shared style rules
- Submit professional-quality programs

Clean code = easier debugging, easier marking, easier teamwork.

Indentation (Very Important)

Python uses indentation to define blocks.

Rule:

- Use **4 spaces**
- Do NOT mix tabs and spaces

```
if mark >= 50:
```

```
    print("Pass")
```

```
else:
```

```
    print("Fail")
```

Line Length

Rule:

- Maximum **79 characters per line**
- Break long lines using parentheses

```
total = (price * quantity
```

```
    + vat
```

```
    - discount)
```

This improves readability.

Blank Lines

Use blank lines to separate logic sections.

Rule:

- 1 blank line between methods inside a class
- 2 blank lines between functions

```
def calculate_total():
```

```
    pass
```

```
def print_invoice():
```

pass

Naming Conventions

This is commonly tested and marked.

Variables

Use **lowercase with underscores**

```
student_mark = 75
```

```
total_price = 500
```

Wrong:

```
StudentMark
```

```
studentMark
```

Functions

Also use **lowercase_with_underscores**

```
def calculate_average():
```

pass (the pass keyword tells Python that it should notice the function definition, but the implementation will follow later)

Classes

Use **PascalCase (CapWords)**

```
class StudentRecord:
```

pass (the pass keyword tells Python that it should notice the Class definition, but the implementation will follow later)

Constants

Use **UPPERCASE**

```
MAX_MARK = 100
```

```
VAT_RATE = 0.15
```

Spaces Around Operators

Always use spaces around operators:

```
x = 5 + 3
```

```
total = price * quantity
```

Avoid:

```
x=5+3
```

Comments

Comments explain *why*, not what.

Good:

```
# Calculate final mark including bonus
```

```
final_mark = mark + bonus
```

Bad:

```
# Add mark and bonus  
final_mark = mark + bonus
```

Imports

Imports go at the top of the file
One import per line

```
import math  
import random
```

Avoid Long or Complex Lines

Instead of this:

```
if mark >= 50 and attendance >= 80 and assignment_submitted == True:
```

Prefer:

```
passed_exam = mark >= 50  
good_attendance = attendance >= 80  
submitted = assignment_submitted  
if passed_exam and good_attendance and submitted:
```

This improves clarity

Don't Compare to True or False**Avoid:**

```
if result == True:
```

Prefer:

```
if result:
```

Use Meaningful Names**Poor:**

```
x = 45  
y = 12
```

Better:

```
exam_mark = 45  
bonus_mark = 12
```

What NQF Level 6 Students Must Demonstrate

At this level, students should:

- Apply PEP 8 consistently
- Use correct naming conventions

- Structure programs clearly
- Write readable and maintainable code
- Avoid sloppy formatting

Useful Tools (Optional for Students)

- black (automatic formatter)
- flake8 (style checker)
- VS Code auto-format feature

When marking code, the following will be checked (this includes code that you have to write for online assessments).

- 4-space indentation
- Correct variable naming
- Proper function naming
- Class names in PascalCase
- Clear spacing around operators
- Logical use of blank lines
- Meaningful variable names
- No excessive line length